COMPTE-RENDU
Analyse de conversations patient-thérapeute sur la santé mentale
Étape 1 — Exploration du jeu de données

Auteur : LARROUX Logan
Encadrante : Lynda Tamine-Lechani

10 avril 2026

[Info5.BE-SHS]

Données : Kaggle — NLP Mental Health Conversations

Code : GitHub — branche Logan

Table des matières

1 Introduction **3**

2 Pré-traitement des données **3**
2.1 Nettoyage . . . 3
2.2 Tokenisation . . . 4
2.3 Suppression de la ponctuation et des mots vides . . . 4
2.4 Lemmatisation . . . 4

3 Analyse statistique **5**
3.1 Nombre de patients et thérapeutes uniques . . . 5
3.2 Nombre de mots moyens par conversation . . . 5
3.3 Distribution des conversations par thérapeutes . . . 5
3.4 Nombre d'échanges moyen par conversations . . . 6
3.5 Mots les plus fréquents . . . 7
3.6 Corrélation patient / thérapeute . . . 8
3.7 Hapax legomena . . . 9

4 Analyse lexicale — N-grams **10**
4.1 Définition . . . 10
4.2 Choix du n optimal . . . 10
4.3 Résultats . . . 11

5 Modélisation et extraction de sujets (Topic Modeling) **12**
5.1 Méthode 1 — Filtrage par lexique métier . . . 12
5.1.1 Principe . . . 12
5.1.2 Résultats et limites . . . 12
5.2 Méthode 2 — Vectorisation TF-IDF suivie d'un clustering K-Means . . . 14
5.2.1 TF-IDF . . . 14
5.2.2 Vectorisation TF-IDF . . . 15
5.2.3 Clustering K-Means . . . 15
5.2.4 Résultats[1] . . . 16
5.3 Méthode 3 — Word Embedding (Word2Vec) couplé à K-Means . . . 17
5.3.1 Principe du Word Embedding . . . 17
5.3.2 Skip-Gram vs CBOW . . . 17
5.3.3 Paramètres du modèle . . . 17
5.3.4 Résultats . . . 18
5.4 Méthode 4 — Modèle probabiliste LDA . . . 19

1. à noter que les thèmes décrit pour les patients et thérapeutes sont des observations visuelles et non généré par le modèle, celui-ci génère uniquement les groupes de mots.

5.4.1 Principe 19
5.4.2 Résultats — Patients 19
5.4.3 Résultats — Thérapeutes 19
5.4.4 Résultats — Global 20

6 Comparaison des méthodes **21**
6.1 Conclusion 21

7 Conclusion générale **22**

1 Introduction

La santé mentale est un enjeu majeur de la santé publique. Ce projet vise, dans un premier temps, à exploiter le jeu de données pour comprendre les types de conversations et leur structure, puis à analyser les sentiments des échanges afin de détecter les émotions et états psychologiques des patients. Ensuite, il se concentre sur l'accès et le traitement des conversations, en résumant les échanges et en identifiant les schémas linguistiques récurrents. Enfin, il prévoit le développement d'un système question-réponse, capable d'automatiser les interactions et fournir des réponses.

Le jeu de données utilisé est composé de deux colonnes : `Context` (messages du patient) et `Response` (réponses du thérapeute).

2 Pré-traitement des données

2.1 Nettoyage

Avant toute analyse, plusieurs étapes de nettoyage ont été appliquées dans l'ordre suivant.

Suppression des cellules vides et des doublons. Les lignes contenant des valeurs nulles dans les colonnes `Context` ou `Response` ont été supprimées, ainsi que les lignes entièrement dupliquées.

Filtrage des textes non-anglais. Le corpus contenait des phrases en espagnol. La bibliothèque `langdetect` a été utilisée pour détecter la langue de chaque ligne et ne conserver que les textes identifiés comme anglais. Cette détection est effectuée au niveau de la phrase entière, ce qui est bien plus fiable qu'une détection mot par mot.

```
from langdetect import detect, LangDetectException

def is_english_text(text):
    try:
        return detect(str(text)) == 'en'
    except LangDetectException:
        return True  # texte trop court -> on garde

mask = df['Context'].apply(is_english_text) \
     & df['Response'].apply(is_english_text)
df = df[mask].reset_index(drop=True)
```

Normalisation. Tous les textes ont été passés en minuscules afin d'éviter que *"Toto"* et *"toto"* soient traités comme deux mots distincts. Les espaces insécables (`\xa0`), fréquents dans des données extraites du web, ont également été supprimés.

2.2 Tokenisation

La tokenisation consiste à découper chaque phrase en unités élémentaires appelées *tokens*. La bibliothèque **spaCy** (`en_core_web_sm`) a été utilisée pour cette étape, via `nlp.pipe()` afin de traiter les données en batch et réduire le temps de traitement.

2.3 Suppression de la ponctuation et des mots vides

Les *stopwords* (mots vides) sont des mots sans réelle signification sémantique (pronoms, adverbes, mots de liaison). spaCy les identifie automatiquement via l'attribut `token.is_stop`. Les tokens de ponctuation ont été supprimés de la même façon avec `token.is_punct`.

Des mots trop génériques mais non reconnus comme stopwords ont également été filtrés manuellement :

```
extra_stopwords = {
    "feel", "like", "want", "know", "time",
    "thing", "good", "need", "year", "think",
    "tell", "say", "come", "go"
}
```

2.4 Lemmatisation

La lemmatisation transforme chaque mot en sa forme de base (*"running"* → *"run"*, *"better"* → *"good"*). Cette étape réduit drastiquement le nombre de tokens distincts et regroupe les formes similaires. Elle a également permis de filtrer les URLs, emails et chiffres dans une seule passe :

```
df["Context_clean"] = [
    [
        token.lemma_
        for token in doc
        if not token.is_stop
        and not token.is_punct
        and not token.is_space
        and not token.like_num
        and not token.like_url
        and not token.like_email
        and token.is_alpha
        and token.lemma_ not in extra_stopwords
    ]
    for doc in docs_context
]
```

Note : spaCy lemmatise différemment selon le contexte grammatical. Par exemple, "feeling" utilisé comme nom reste tel quel au lieu d'être ramené à "feel". C'est pourquoi "feel" a été retiré manuellement des extra_stopwords pour éviter les doublons sémantiques.

3 Analyse statistique

3.1 Nombre de patients et thérapeutes uniques

Il n'existe pas vraiment de façon assez fiable pour différencier un patient/thérapeute d'un autre, la solution aurait été d'avoir un identifiant unique pour chaque patient/thérapeutes. En l'absence de cela dans le dataset, le nombre de patients uniques est estimé par le nombre de valeurs distinctes dans la colonne `Context`, et de même pour les thérapeutes avec `Response`. Cette méthode reste approximative.

3.2 Nombre de mots moyens par conversation

Le tableau suivant compare la longueur moyenne des messages avant et après lemmatisation :

Mesure	**Patients**	**Thérapeutes**
Avant nettoyage	820	819
Après nettoyage	2405	1985

Table 1 – Longueur moyenne des messages avant et après nettoyage

3.3 Distribution des conversations par thérapeutes

On cherche ici combien de conversations peut avoir un thérapeutes avec un ou plusieurs patients. Or le problème ici est que, comme mentionné précedemment (section 3.1), il n'existe pas de moyen fiable pour différencier les patients et les thérapeutes, et donc aucuns moyens de déterminer le nombre de conversations par thérapeutes.

3.4 Nombre d'échanges moyen par conversations

En supposant que tout les patients et thérapeutes sont uniques, on va chercher le nombre de **mot moyens par patients/thérapeutes** sous plusieurs conditions : avec les *stopwords*, sans les *stopwords* et avec la lemmatisation.

Mesure	Patients	Thérapeutes
Avec *stopwords*	290.77	1011.24
Sans *stopwords*	26.55	87.95
Après lemmatisation	16.29	59.69

TABLE 2 – Nombre de mots moyens par patients/thérapeutes selon les différentes étapes de nettoyage

On peut donc constater que :

- **avec les stopwords**, les thérapeutes utilisent plus de mots que les patients (**28.75% de plus**).
- **après lemmatisation**, les thérapeutes utilisent plus de mots que les patients (**27.29% de plus**).

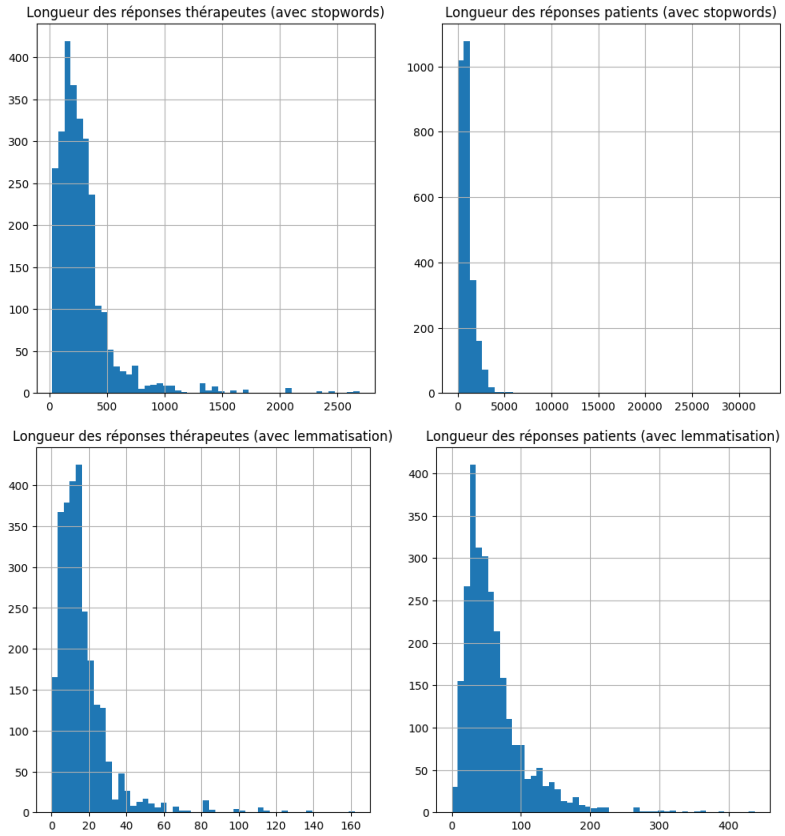

FIGURE 1 – Distribution des longueurs des messages des thérapeutes avant et après lemmatisation

3.5 Mots les plus fréquents

En précisant que l'on a retiré des stopwords supplémentaires (mentionnée dans la section 2.3), on peut constater que les mots les plus fréquents sont des termes génériques liés à la santé mentale, tels que *"anxiety"*, *"depression"*, *"help"*, *"family"*, *"relationship"*, etc. Ces mots reflètent les thèmes centraux abordés dans les conversations, une information supplémentaire qui nous sera utile dans les modèles.

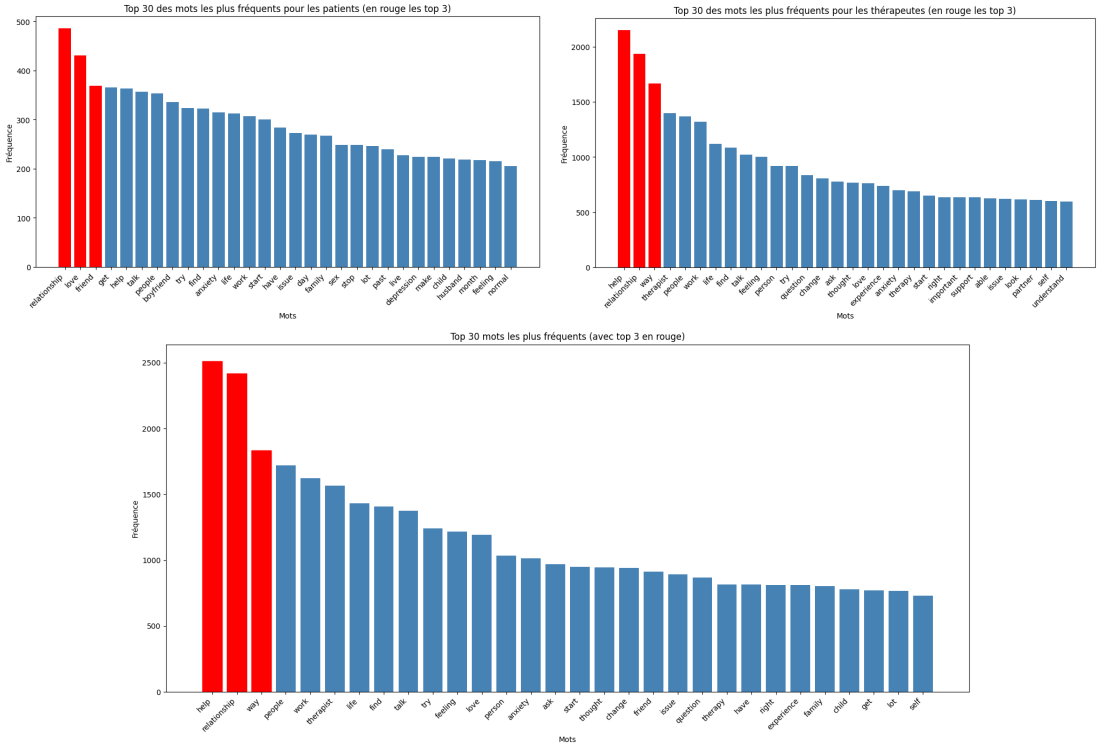

FIGURE 2 – Mots les plus fréquents chez les patients/thérapeutes et globalement

Si on aurait gardé *feel* des mots à analyser, on aurait remarqué ici que *feel* et *feeling* sont deux termes distincts, alors que leur sens est très proche. Cela vient du fait que spaCy **lemmatise différement selon le contexte grammatical** du mot. Quand feeling est utilisé comme nom (*"a feeling of anxiety"*), spaCy le garde tel quel au lieu de le remener à feel.

Ensuite, *like* est un mot ambigue en anglais, il peut être un verbe (*"I like this"*) ou une préposition (*"feel like crying"*), auquel cas c'est quasiment un mot vide d'où sa suppression.

3.6 Corrélation patient / thérapeute

La correlation mesure, dans notre cas, le lien qu'il peut y avoir entre la longueur des phrases du patient et du thérapeute. Cela nous permet de voir si les thérapeutes adaptent la longueur de leurs réponses à celle des messages des patients, ou si au contraire ils utilisent des réponses de longueur relativement constante indépendamment de la longueur du message patient.

Le résultat obtenu est une **corrélation proche de 0** (≈ 0.05), cela signifie qu'**un patient peut s'exprimer longuement et recevoir une réponse courte, et inversement**. Les réponses des thérapeutes semblent donc relativement indépendantes de la longueur des messages patients, ce qui peut s'expliquer par l'utilisation de réponses standardisées ou formulées de façon uniforme.

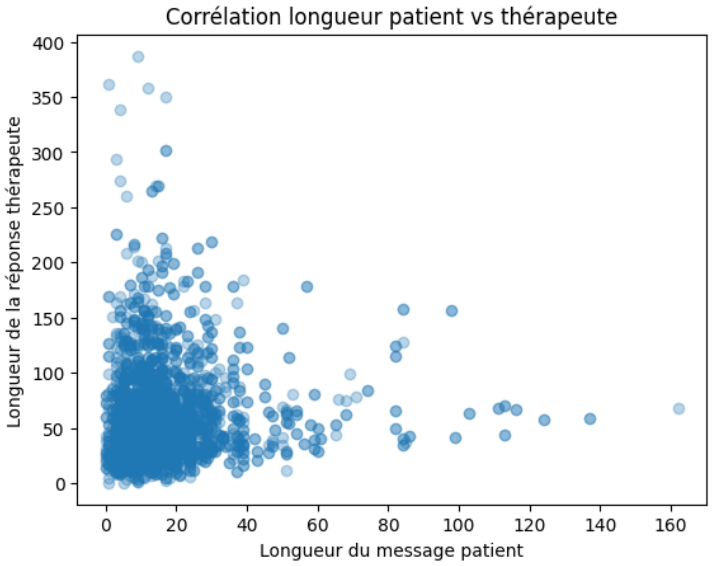

FIGURE 3 – Nuage de points représentant la longueur des messages patients (en abscisse) et thérapeutes (en ordonnée)

3.7 Hapax legomena

Un *hapax* est un mot qui n'apparaît qu'**une seule fois dans le corpus**. L'analyse des mots les moins fréquents a révélé que la plupart des hapax sont du **bruit résiduel** (abréviations, artefacts de nettoyage ou même mot en espagnol passé à travers les mailles du fillet), ce qui confirme l'importance du filtrage préalable.

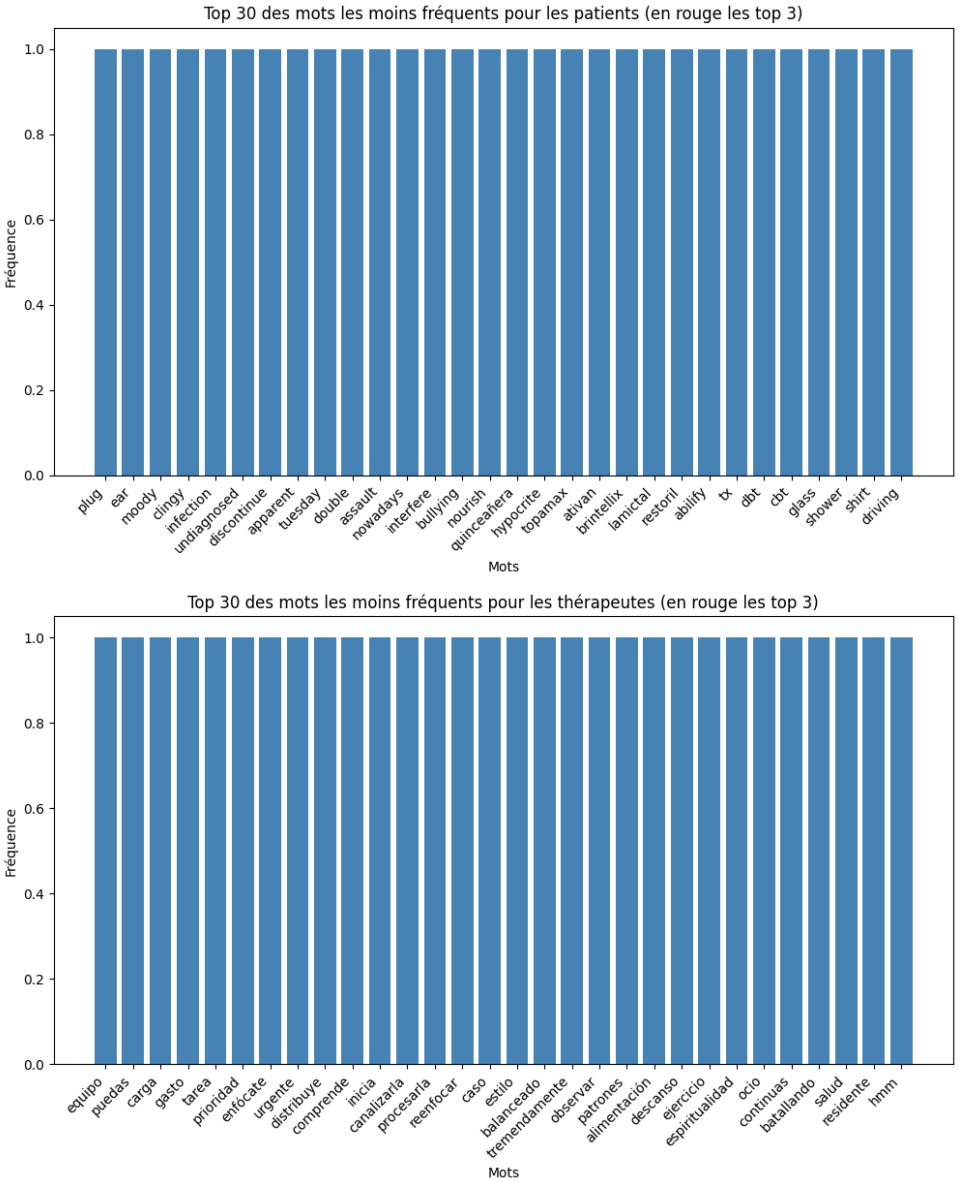

FIGURE 4 – Distribution des hapax dans le corpus

4 Analyse lexicale — N-grams

4.1 Définition

Un **n-gram** est une séquence de n mots consécutifs dans un texte. Cette approche permet de capturer non seulement les mots isolés, mais aussi les associations de mots fréquentes.

- **Unigram (n=1)** : mots individuels, ex. *"anxiety"*, *"depression"*.
- **Bigram (n=2)** : paires de mots, ex. *"mental health"*, *"feel like"*.
- **Trigram (n=3)** : triplets de mots, ex. *"feel like cry"*, *"family support system"*.
- ainsi de suite...

Dans notre travail, nous allons devoir repérer des formulations récurrentes qui ont du sens :

Type	Exemple	Interprétation
Unigram (n=1)	*anxiety* (fréq. 342)	Thème dominant
Bigram (n=2)	*mental health* (fréq. 87)	Expression caractéristique
Trigram (n=3)	*feel like cry* (fréq. 54)	Patron de détresse

Table 3 – Exemples de n-grams et leur interprétation

4.2 Choix du n optimal

Il n'existe pas de valeur universelle pour n. Deux indicateurs ont été utilisés pour guider ce choix :

- **Fréquence moyenne** : si elle chute sous un seuil de 5, les n-grams sont trop rares pour être statistiquement significatifs.
- **Pourcentage d'hapax** : si plus de 70 % des n-grams n'apparaissent qu'une seule fois, la taille n est trop grande pour le corpus.

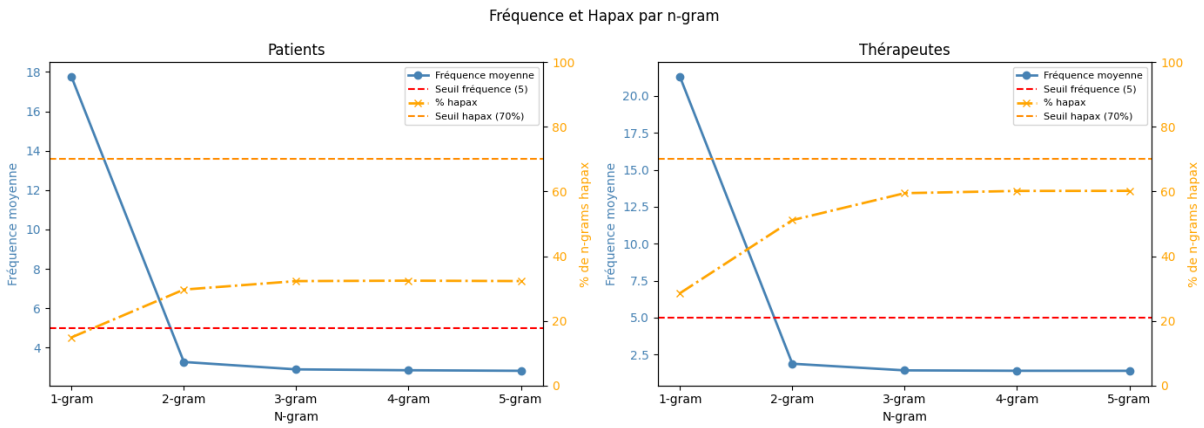

Figure 5 – Graphe de fréquence moyenne et pourcentage d'hapax en fonction de n

On remarque qu'à partir de $n = 1$, la fréquence diminue déjà fortement. Cependant, les bigrams et trigrams restent exploitables car le seuil d'hapax n'est pas dépassé.

4.3 Résultats

Les bigrams et trigrams apportent un contexte nettement plus riche que les unigrams seuls. Ils peuvent permettre par la suite d'identifier des patrons linguistiques et des thèmes récurrents pour le topic modeling mais malheureusement leur fréquence est relativement faible, ce qui rend leur exploitation plus délicate...

Unigram	**fréquence**	**Unigram**	**fréquence**
relationship	486	*help*	2150
love	431	*relationship*	1933
friend	369	*way*	1668

Table 4 – Trois plus fréquent **unigram** pour les patients (gauche) et thérapeutes (droite)

Bigram	**fréquence**	**Bigram**	**fréquence**
issue address	94	*therapist help*	160
self esteem	65	*ask question*	154
month ago	64	*mental health*	153

Table 5 – Trois plus fréquent **bigram** pour les patients (gauche) et thérapeutes (droite)

Trigram	**fréquence**	**Trigram**	**fréquence**
history sexual abuse	51	*hello thank question*	68
low self esteem	50	*mental health professional*	52
couple therapy session	48	*landwehr dbh lpc*	51

Table 6 – Trois plus fréquent **trigram** pour les patients (gauche) et thérapeutes (droite)

5 Modélisation et extraction de sujets (Topic Modeling)

Quatre méthodes ont été implémentées et comparées pour identifier les thèmes latents dans les conversations.

5.1 Méthode 1 — Filtrage par lexique métier

5.1.1 Principe

La première méthode consiste à définir **manuellement** un dictionnaire où chaque clé est un thème et chaque valeur est une liste de mots associés à ce thème.

```
lexique = {
    "anxiété":    ["anxiety", "anxious", "worry", "nervous", "
       panic", "fear", "stress"],
    "dépression": ["depression", "depressed", "sad", "hopeless",
       "empty", "worthless"],
    "sommeil":    ["sleep", "insomnia", "nightmare", "tired", "
       fatigue", "wake"],
    "trauma":     ["trauma", "abuse", "ptsd", "flashback", "
       violent", "assault"],
    "relations":  ["relationship", "family", "friend", "lonely",
       "isolate", "divorce"],
}
```

5.1.2 Résultats et limites

L'analyse a révélé que les thérapeutes expriment plus souvent des termes liés aux relations familiales et amicales que les patients eux-mêmes.

Cependant, cette méthode présente des **limites importantes** :

- Le dictionnaire est subjectif et incomplet : des mots comme *"feeling"* ou *"down"* n'appartiennent à aucun thème.
- Elle ne tient pas compte du contexte : *"down"* peut relever de l'anxiété ou de la dépression selon la phrase.
- Elle est statique : elle ne s'adapte pas au vocabulaire réel du corpus.

Thème	Mot	fréquence	Mot	fréquence
Anxiété	*anxiety*	315	*anxiety*	700
	stress	169	*fear*	423
	nervous	84	*stress*	300
Dépression	*depression*	224	*depression*	393
	depressed	122	*sad*	120
	sad	88	*depressed*	54
Sommeil	*sleep*	139	*trauma*	205
	nightmare	45	*abuse*	188
	tired	44	*ptsd*	63
Relations	*relationship*	486	*relationship*	1933
	friend	369	*friend*	544
	family	267	*family*	536

TABLE 7 – Trois mots les plus fréquents pour les patients (gauche) et thérapeutes (droite) selon le thème

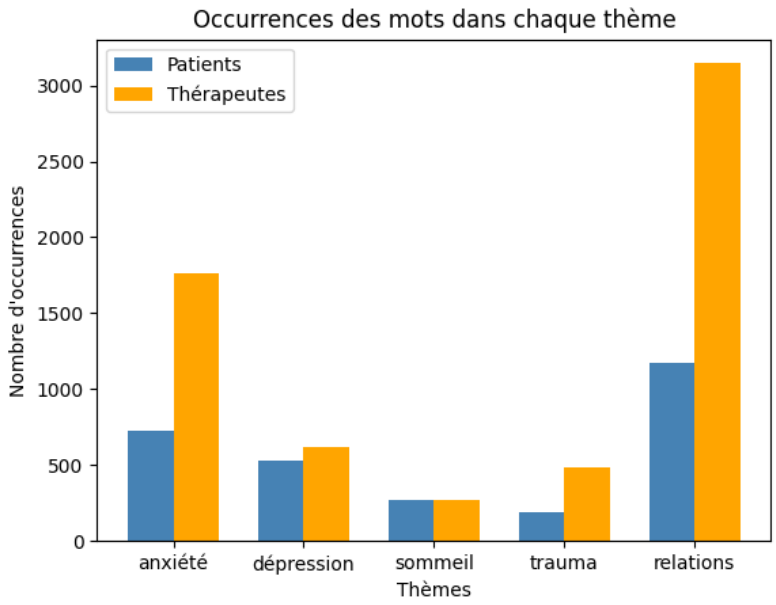

FIGURE 6 – Distribution des hapax dans le corpus

On peut remarquer que depuis le graphe que les thérapeutes vont oser exprimer beaucoup plus souvent des termes liées à notre dictionnaire, en particulier les termes liées aux **relations famillialle et amicale**, une observation intéressante mais **très limitée**.

5.2 Méthode 2 — Vectorisation TF-IDF suivie d'un clustering K-Means

5.2.1 TF-IDF

Le **TF-IDF** (*Term Frequency — Inverse Document Frequency*) est une méthode de pondération évaluant l'importance d'un mot au sein d'un document par rapport à l'ensemble du corpus. Il combine deux composantes :

— **Term Frequency (TF)** : fréquence d'un terme dans le document.

$$tf_{i,j} = \frac{n_{i,j}}{\sum_k n_{k,j}}$$

où $n_{i,j}$ est le nombre d'occurrences du terme i dans le document j.

— **Inverse Document Frequency (IDF)** : pondération par la rareté du mot dans le corpus.

$$idf(w) = \log\left(\frac{N}{df_t}\right)$$

où N est le nombre total de documents et df_t le nombre de documents contenant le terme t.

Le score final est :

$$w_{i,j} = tf_{i,j} \times \log\left(\frac{N}{df_i}\right)$$

Les vecteurs sont ensuite normalisés par la norme L2 afin que la longueur des documents ne biaise pas les comparaisons.

Grâce au pré-traitement déjà effectué (suppression des stopwords, lemmatisation), l'IDF discrimine mieux les termes et les scores sont plus significatifs.

5.2.2 Vectorisation TF-IDF

Nous allons utiliser la fonction `TfidfVectorizer` de la bibliothèque **scikit-learn**. Cette fonction à de grand avantages, elle permet de :

- lister tout les mots uniques
- définir le score TF
- définir le score IDF
- définir le score final et vecteur L2

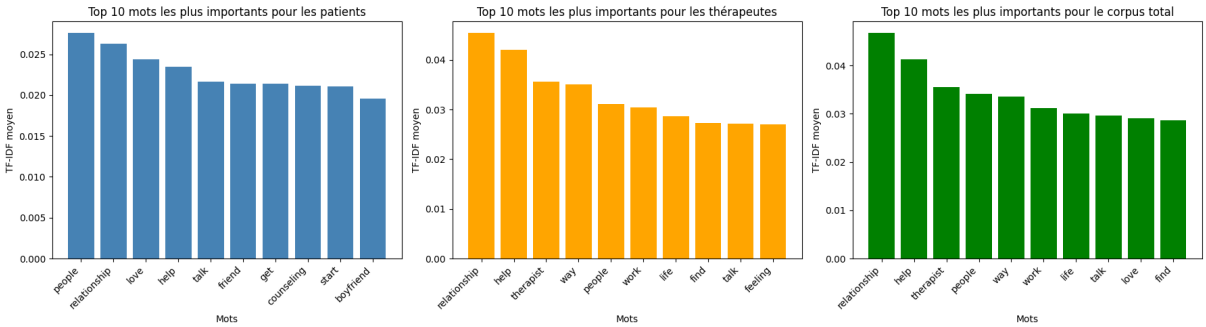

FIGURE 7 – Histogramme des 10 mots les plus importants selon le score TF-IDF

5.2.3 Clustering K-Means

Nous allons devoir maintenant regroupper les mots similaires en clusters, or nous devons dans un premier temps définir le nombre de clusters K à utiliser. Un des risques est de choisir un K trop petit ou trop grand, ce qui peut conduire à des clusters peu significatifs ou à un sur-apprentissage. La méthode du coude (*elbow method*) a été utilisée pour déterminer le nombre optimal de clusters K. L'inertie diminue fortement jusqu'à $K = 6$, puis la courbe se stabilise. **K = 6 a donc été retenu.**

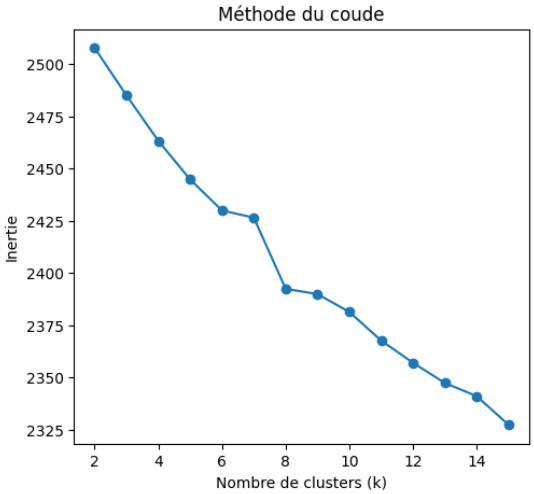

FIGURE 8 – Graphe de la méthode du coude pour déterminer le nombre optimal de clusters K

5.2.4 Résultats [1]

Clusters	Patient	Thérapeute	Global
Cluster 0	Dépression sévère — idées suicidaires	Soutien initial — écoute	Dépression sévère → écoute
Cluster 1	Mal-être général — perte de motivation	Accompagnement émotionnel	Mal-être → accompagnement émotionnel
Cluster 2	Dépression sévère — fatigue mentale	Conseils pratiques	Dépression → conseils pratiques
Cluster 3	Thérapie de couple — anxiété	Accompagnement en couple	Couple → thérapie
Cluster 4	Traumatisme — abus — maladie grave	Suivi structuré	Traumatisme → suivi structuré
Cluster 5	Problèmes familiaux — divorce — identité	Soutien familial	Famille → soutien familial

TABLE 8: Résultats TF-IDF + K-Means ($K = 6$)

Les clusters sont basés sur des similarités lexicales. On note un léger chevauchement entre les clusters 0 et 2 (dépression), **ce qui peut s'interpréter positivement** : la dépression peut provenir de sources différentes (fatigue mentale vs. idées suicidaires). Ce détail révèle que **les conversations ne sont pas aléatoires** — les thérapeutes adaptent leur stratégie de réponse en fonction du type de détresse exprimée par le patient.

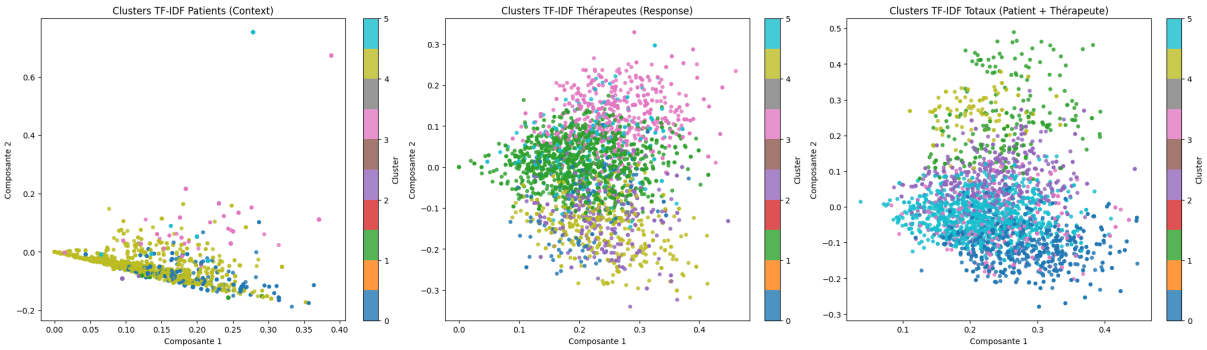

FIGURE 9 – Projection en 2 dimenssions des conversations dans l'espace TF-IDF avec les clusters K-Means colorés

Attention, le plan est applatit sur deux dimenssions, ce qui peut masquer des nuances présentes dans les dimensions originales.

1. à noter que les thèmes décrit pour les patients et thérapeutes sont des observations visuelles et non généré par le modèle, celui-ci génère uniquement les groupes de mots.

5.3 Méthode 3 — Word Embedding (Word2Vec) couplé à K-Means

5.3.1 Principe du Word Embedding

Le **Word Embedding** (plongement lexical) est une représentation sémantique des mots sous forme de vecteurs de nombres réels. Deux mots de sens proche ont des vecteurs proches dans l'espace mathématique.

Contrairement à TF-IDF qui est une **représentation statistique** (fréquence d'apparition), Word2Vec est une **représentation sémantique** : il apprend ce que le mot signifie en observant les mots qui l'entourent dans le corpus. Ainsi, *"sad"* et *"depressed"* se trouvent proches dans l'espace vectoriel parce qu'ils apparaissent dans des contextes similaires.

5.3.2 Skip-Gram vs CBOW

Word2Vec propose deux architectures :

- **CBOW** (*Continuous Bag of Words*) : prédit le mot central à partir de son contexte. Très rapide sur les grands corpus, mais la moyenne du contexte *lisse le bruit*, ce qui peut effacer des nuances importantes.
- **Skip-Gram** (`sg=1`) : prédit le contexte à partir du mot central. Plus adapté aux **corpus petits et spécialisés** car il génère plus d'exemples d'entraînement par mot, compensant le manque de données.

Skip-Gram a été retenu pour ce projet, le corpus étant de taille modeste et spécialisé dans le domaine thérapeutique.

5.3.3 Paramètres du modèle

```
model = Word2Vec(
    sentences=all_sentences,
    vector_size=100,    # taille des vecteurs
    window=5,           # contexte de 5 mots de chaque côté
    min_count=3,        # ignore les mots apparaissant moins de 3
       fois
    workers=4,
    epochs=10,
    seed=42,
    sg=1                # Skip-Gram
)
```

Chaque conversation est vectorisée en calculant la **moyenne des vecteurs** de ses tokens présents dans le vocabulaire. Les vecteurs patients et thérapeutes sont ensuite fusionnés par moyenne pour obtenir un vecteur par échange.

5.3.4 Résultats

Clusters	Patient	Thérapeute	Global
Cluster 0	Dépression sévère — idées suicidaires	Soutien émotionnel	Dépression → soutien
Cluster 1	Problèmes sexuels — santé physique	Orientation médicale	Physique → médical
Cluster 2	Thérapie de couple — anxiété	Accompagnement en couple	Couple → thérapie
Cluster 3	Mal-être quotidien — divorce	Introspection — gestion émotionnelle	Mal-être → introspection
Cluster 4	Anxiété — troubles du sommeil	Conseils thérapeutiques	Anxiété → gestion
Cluster 5	Traumatisme — abus	Accompagnement structuré	Traumatisme → suivi

TABLE 9: Résultats Word2Vec + K-Means ($K = 6$)

Par rapport à TF-IDF, Word2Vec offre une **meilleure séparation des thèmes**, notamment pour les patients où la dépression est regroupée dans un seul cluster au lieu de deux. La cohérence entre les problématiques des patients et les stratégies des thérapeutes est également plus lisible.

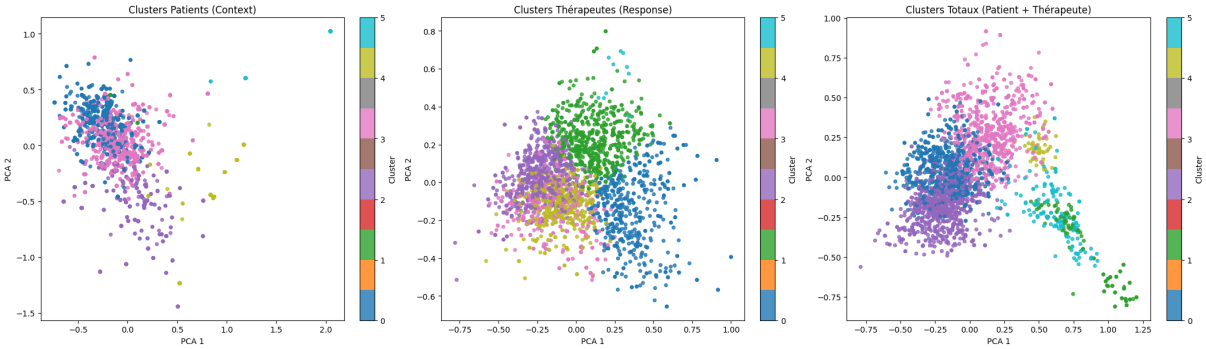

FIGURE 10 – Projection en 2 dimenssions des conversations dans l'espace Word2Vec avec les clusters K-Means colorés

5.4 Méthode 4 — Modèle probabiliste LDA

5.4.1 Principe

Le **LDA** (*Latent Dirichlet Allocation*) est un modèle génératif probabiliste permettant de découvrir des sujets latents dans un corpus. C'est une approche bayésienne qui suppose que chaque document est un **mélange de thèmes**, et que chaque thème est une **distribution de mots**.

Contrairement à K-Means où chaque document appartient à un unique cluster, LDA assigne à chaque conversation un mélange de thèmes avec des probabilités. Par exemple :

$$\textit{"I feel sad and anxious"} \longrightarrow 70\,\%\ \text{dépression} + 30\,\%\ \text{anxiété}$$

LDA produit deux sorties :

1. **Thèmes → mots** : distribution des mots caractéristiques pour chaque thème.
2. **Documents → thèmes** : proportion de chaque thème dans chaque conversation.

5.4.2 Résultats — Patients

Topic	Nom suggéré	Mots dominants
Topic 0	Famille — vie quotidienne — anxiété	family, life, anxiety, parent, child
Topic 1	Dépression — addictions — aide	depression, help, substance, hopeless
Topic 2	Relations sociales — pensées	relationship, thought, people, past
Topic 3	Thérapie de couple — anxiété	couple, anxiety, partner, therapist
Topic 4	Sexualité — estime de soi	sexual, esteem, body, love
Topic 5	Relations amoureuses	relationship, boyfriend, trust, person

Table 10 – Topics LDA — Patients

5.4.3 Résultats — Thérapeutes

Topic	Nom suggéré	Mots dominants
Topic 0	Suivi thérapeutique	therapist, help, work, session
Topic 1	Parentalité — conseils familiaux	parent, child, family, talk
Topic 2	Relations de couple	relationship, partner, love, work
Topic 3	Bruit résiduel	(mots peu cohérents)
Topic 4	Gestion des pensées — émotions	thought, feeling, way, manage
Topic 5	Dépression — accompagnement	depression, anxiety, help, support

Table 11 – Topics LDA — Thérapeutes

5.4.4 Résultats — Global

Topic	Nom suggéré	Mots dominants
Topic 0	Santé / sexualité / aide	health, help, sexual, anxiety
Topic 1	Communication — écoute	talk, people, way, understand
Topic 2	Anxiété — pensées — soutien	anxiety, thought, help, support
Topic 3	Famille — enfants	family, parent, child, live
Topic 4	Thérapie / counseling	therapist, therapy, work, session
Topic 5	Relations de couple	relationship, boyfriend, partner, trust

Table 12 – Topics LDA — Global

LDA a permis d'identifier des thématiques principales cohérentes avec le domaine étudié : relations de couple, parentalité, troubles anxieux et suivi thérapeutique. La flexibilité du modèle probabiliste permet à chaque conversation d'appartenir à plusieurs thèmes simultanément, ce qui est plus fidèle à la réalité des échanges thérapeutiques.

6 Comparaison des méthodes

Méthode	Avantages	Inconvénients	Usage
TF-IDF + K-Means	Simple, rapide, bonne première approche	Ignore le sens des mots ; clusters parfois redondants	Exploration initiale
Word2Vec + K-Means	Capture les relations sémantiques ; meilleure qualité de clustering ; thèmes plus fins	Plus complexe ; résultats moins interprétables directement	Regroupement sémantique
LDA	Thèmes interprétables ; modèle probabiliste flexible ; un document peut appartenir à plusieurs thèmes	Sensible au prétraitement ; peut produire du bruit ; choix de K délicat	Analyse thématique
Lexique métier	Très simple ; résultats immédiatement lisibles	Subjectif, figé, incomplet	Introduction / baseline

Table 13 – Comparaison des quatre méthodes de topic modeling

6.1 Conclusion

Ces méthodes sont **complémentaires plutôt que concurrentes**.

- Le **LDA** est le plus pertinent pour l'**analyse thématique** : il produit des sujets clairement interprétables et cohérents avec le domaine.
- **Word2Vec** offre la **meilleure qualité de clustering** en capturant les similarités sémantiques.
- **TF-IDF** reste utile comme méthode de référence rapide.
- Le **lexique métier** constitue une bonne introduction mais ses résultats sont trop limités pour une analyse approfondie.

7 Conclusion générale

Cette première étape a permis de comprendre la structure du corpus et d'extraire les thèmes principaux des conversations patient-thérapeute. Le pré-traitement rigoureux (filtrage des langues, lemmatisation, stopwords) a été déterminant pour la qualité des résultats de topic modeling.

Les analyses confirment que les conversations ne sont pas aléatoires : les thérapeutes adaptent leurs stratégies de réponse en fonction des problématiques exprimées par les patients. La prochaine étape consistera à analyser les sentiments des échanges afin de quantifier la polarité émotionnelle des conversations.

Références

[1] Kaggle, *NLP Mental Health Conversations*, https://www.kaggle.com/datasets/thedevastator/nlp-mental-health-conversations

[2] GitHub, *Dépôt du projet – branche Logan*, https://github.com/cypnet/Analyse-de-conversations-sur-la-sant-mentale-/tree/Logan

[3] Wikipédia, *TF-IDF*, https://fr.wikipedia.org/wiki/TF-IDF

[4] Wikipédia, *Word Embedding*, https://fr.wikipedia.org/wiki/Word_embedding

[5] Wikipédia, *Allocation de Dirichlet Latente*, https://fr.wikipedia.org/wiki/Allocation_de_Dirichlet_latente

[6] Wikipédia, *Mots vides*, https://fr.wikipedia.org/wiki/Mot_vide

[7] Wikipédia, *Hapax*, https://fr.wikipedia.org/wiki/Hapax

[8] freeCodeCamp, *How to Process Textual Data Using TF-IDF in Python*, https://www.freecodecamp.org/news/how-to-process-textual-data-using-tf-idf-in-python-cd2bbc0a94a3

[9] GeeksforGeeks, *Word Embeddings in NLP*, https://www.geeksforgeeks.org/nlp/word-embeddings-in-nlp/

[10] GeeksforGeeks, *N-gram in NLP*, https://www.geeksforgeeks.org/nlp/n-gram-in-nlp/

[11] IBM, *Latent Dirichlet Allocation*, https://www.ibm.com/fr-fr/think/topics/latent-dirichlet-allocation

[12] Spiriit, *Data Clustering*, https://www.spiriit.com/ressources/data-clustering-definition-methodes-usages/

[13] Université Toulouse, *Tutoriel Word Embeddings*, http://w3.erss.univ-tlse2.fr/UETAL/2018-2019/Tuto-embeddings.pdf

[14] Éduscol, *Word Embedding & Machine Learning*, https://sti.eduscol.education.fr/sites/eduscol.education.fr.sti/files/ressources/pedagogiques/14960/14960-word-embedding-les-mots-et-le-machine-learning-ensps.pdf